Architecture 3-tiers

1. Prérequis

● Deux VMs Debian 12 ou supérieure :

● VM Web (172.17.21.X) : installation d'Apache2 et PHP8.

● VM BDD (172.17.21.Y) : installation de MariaDB.

● Un schéma d'architecture doit être réalisé avec IP fixe.

1. Introduction **2**

2. Configuration MariaDB pour l'accès distant **2**

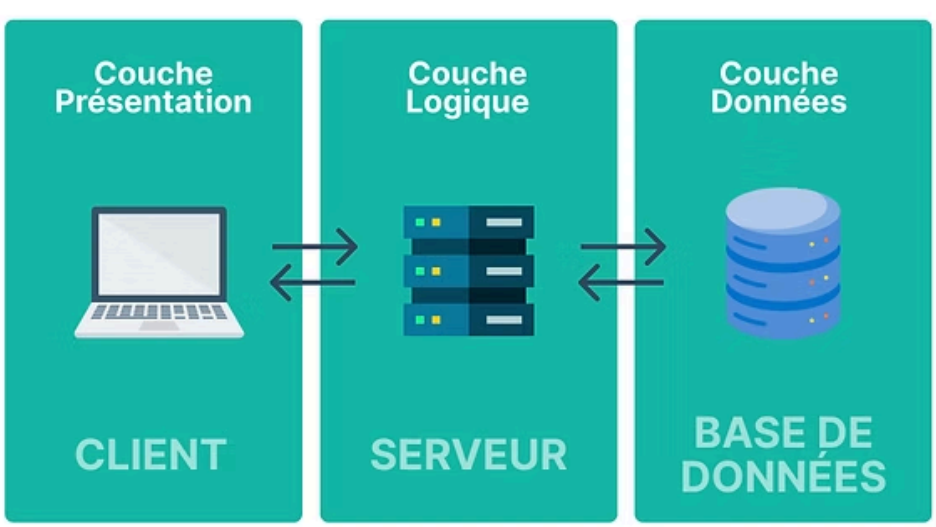
3. Création d'un utilisateur MariaDB **3**

4. Configuration du serveur Web **3**

5. Installation et configuration de phpMyAdmin **4**

Conclusion **6**

OS	Version OS	Date	Version Procédure
Debian	12.5	08/010/2025	V1



1. Introduction

Une architecture 3-tiers (ou architecture à trois niveaux) est un modèle d'architecture logicielle client-serveur qui sépare les fonctions de l'application en trois couches logiques et physiques distinctes, facilitant ainsi le développement, la maintenance et la scalabilité.

Chaque niveau est spécialisé dans une fonction précise et communique uniquement avec les niveaux adjacents :

1. Le niveau présentation (tier présentation)
● Rôle : l'Interface Homme Machine (IHM) qui gère l'affichage des informations et la capture des saisies de l'utilisateur.
● Technologies : navigateur Web (HTML, CSS, JavaScript) ou application cliente lourde.
● Localisation physique : le poste client (ordinateur, smartphone, tablette).

2. Le niveau application (tier application/logique métier)
● Rôle : il contient la logique métier de l'application (règles de gestion, calculs, workflows). Il sert d'intermédiaire entre le niveau présentation et le niveau données.
● Technologies : serveur Web (Apache, Nginx) et langages de programmation côté serveur (PHP, Java/JEE, Python, Node.js).
● Localisation physique : un ou plusieurs serveurs d'application.

3**. Le niveau données (tier données/persistance)**
● Rôle : stockage, gestion et accès aux données. Il est uniquement interrogé par le niveau application.
● Technologies : Système de Gestion de Base de Données (SGBD) comme MariaDB, MySQL, PostgreSQL, Oracle, etc.
● Localisation physique : un ou plusieurs serveurs de base de données.

2. Configuration MariaDB pour l'accès distant

Installer mariadb-server

```
apt install mariadb-server
```

Sur la VM mariaDB, modifier le fichier de configuration :

```
nano  /etc/mysql/mariadb.conf.d/50-server.cnf
```

Remplacer **bind-address** par **0.0.0.0** puis redémarrer :

```
systemctl restart mariadb
```

3. Création d'un utilisateur MariaDB

Lancer mariadb en tapant :

```
mariadb
```

Création de l'utilisateur titi avec le mot passe password123
GRANT ALL PRIVILEGES ON → donne des droits sur la base

```
GRANT ALL PRIVILEGES ON   t_films.* TO 'titi'@'%' IDENTIFIED BY 'password123';
```

Rafraîchir les droits sur la base de donnée

```
FLUSH PRIVILEGES;
```

4. Configuration du serveur Web

Sur la VM Web :

Installer Apache2 et PHP avec le module MySQL :

```
apt install apache2 php libapache2-mod-php php-mysql -y
```

```
php-mysql -y
```

Tester la connexion distante avec :

```
mysql -u titi -h 172.17.21.Y -p
```

5. Installation et configuration de phpMyAdmin

Sur la VM Web :

```
apt install phpmyadmin -y
```

Choisir Apache2 comme serveur web.
Modifier /etc/phpmyadmin/config.inc.php :
Remplacer localhost par 172.17.21.Y
Redémarrer Apache.

```
systemctl restart apache2
```

7. Validation et tests

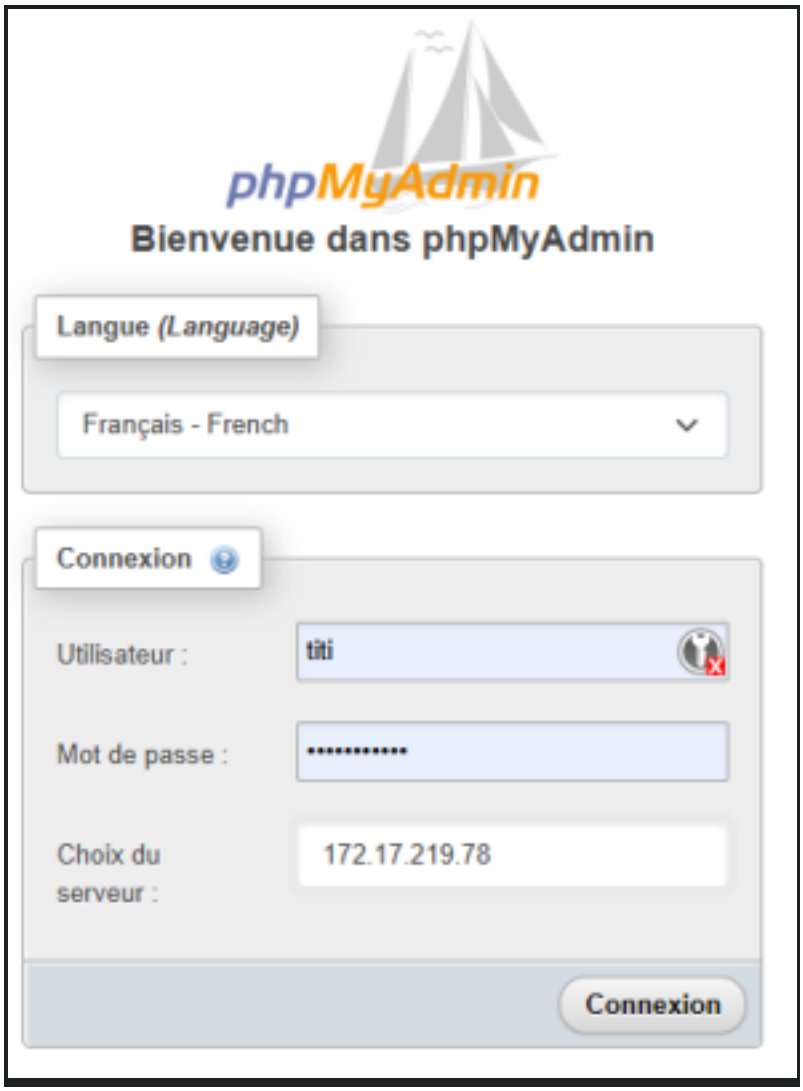
Accéder à phpMyAdmin taper dans l'url client :

```
http://172.17.21.X/phpmyadmin
```

Se connecter avec les identifiants mise en place en amont :

ATTENTION , il faut choisir le bon serveur.

```
Utilisateur : titi
Mot de passe : password123
```



Créer un fichier db_test.php dans /var/www/html/ :

```
<?php
// Configuration de la base de données
$servername = "172.17.21.Y";
$database = "t_films";
$username = "titi";
$password = "password123";
// Connexion à la base de données
$conn = mysqli_connect($servername, $username, $password, $database);
if (!$conn) {
die("Echec de connexion" . mysqli_connect_error());
}
echo "Connexion réussie";
mysqli_close($conn);
?>
```

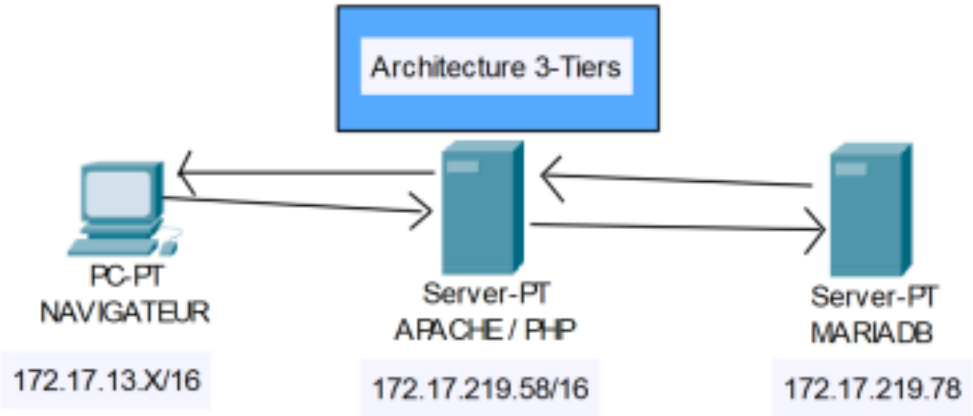
Accéder via : http://172.17.21.X/db_test.php

Si le message 'Connexion réussie' apparaît,l'architecture fonctionne.

Conclusion

Ce TP permet de comprendre et de mettre en place une architecture logicielle en 3-tiers. Il illustre l'importance de la séparation des couches (présentation, application et données) pour améliorer la sécurité, la modularité et la scalabilité.

Schéma d'architecture 3-tiers



Ce TP a permis de concevoir une architecture **3-tiers** en isolant les couches de présentation, d'application (**Apache/PHP**) et de données (**MariaDB**) sur des serveurs distincts. Nous avons configuré l'accès distant à la base de données, sécurisé les privilèges utilisateurs et déployé **phpMyAdmin** pour l'administration. Cette structure modulaire garantit une meilleure sécurité, une maintenance simplifiée et une plus grande flexibilité pour l'évolution des services web.